

UNIVERSIDAD MAYOR REAL Y PONTIFICIA DE SAN FRANCISCO XAVIER



LAB3.1 Infraestructura

Nombre Completo:

Calatayud Mamani Alex Josué

Quispe Anagua Jhon Christian

Chambi Condori Janet

Quispe Sullca Luis Fernando

Introducción

Proxy Inverso y Balanceo de Carga en Servidores Web

En el entorno de servidores web, el uso de herramientas como el proxy inverso y el balanceador de carga es fundamental para mejorar el rendimiento, la seguridad y la disponibilidad de los servicios.

El **proxy inverso** es un servidor intermediario que se sitúa entre los clientes y uno o varios servidores web. Su función principal es recibir las solicitudes de los usuarios y redirigirlas al servidor correspondiente. De esta manera, los servidores reales permanecen ocultos, lo que incrementa la seguridad. Además, el proxy inverso permite gestionar certificados HTTPS, almacenar contenido en caché para acelerar las respuestas y centralizar el acceso a los servicios. Tecnologías como Nginx y Apache HTTP Server son ampliamente utilizadas para este propósito.

Por otro lado, el **balanceador de carga** es un sistema diseñado para distribuir el tráfico de red entre varios servidores. Su objetivo es evitar la sobrecarga de un único servidor, garantizando así un mejor rendimiento y mayor disponibilidad del sistema. En caso de que uno de los servidores falle, el balanceador redirige automáticamente las solicitudes a los servidores disponibles, asegurando la continuidad del servicio. Además, facilita la escalabilidad, ya que permite añadir nuevos servidores según la demanda. Herramientas como HAProxy y también Nginx pueden cumplir esta función.

En conclusión, mientras que el proxy inverso actúa como un intermediario que protege y optimiza el acceso a los servidores, el balanceador de carga se encarga de distribuir eficientemente las solicitudes para mejorar el rendimiento y la estabilidad del sistema.

Metodología

Capturas De Pantalla Detalladas

1. Tablas De Configuración de red (IP, Máscara, Puerta De Enlace)

Cada máquina virtual fue configurada con IP estática. A continuación se detalla la configuración de red completa:

```
GNU nano 8.4 /etc/netu
auto eth0
iface eth0 inet static
    address 192.168.10.3
    netmask 255.255.255.240
    gateway 192.168.10.1
    dns-nameservers 8.8.8.8
```

2. Instalación De Paquetes En Todas Las Maquinas

En todas las máquinas se actualizó el sistema y se instaló NGINX. En los servidores backend se configuraron páginas web de identificación simples para distinguir cuál servidor responde cada petición.

```
webserver4:/var/www/nodejs# nano index.js
webserver4:/var/www/nodejs# npm install

up to date, audited 1 package in 608ms

found 0 vulnerabilities
webserver4:/var/www/nodejs# npm start

> hola-mundo-app@1.0.0 start
> node index.js
```

3. Configuración De NGINX En Cada Servidor

Cada servidor backend (VM2, VM3, VM4) fue configurado con una página HTML de identificación única para verificar visualmente cuál servidor atiende cada petición:

```
GNU nano 8.4 index.js
const http = require('http');
const os = require('os');

// Configuración manual para tu WebServer 3
const PORT = 3000;
const MY_IP = '192.168.10.2';
const MY_HOSTNAME = os.hostname();

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end(`
    <h1>Hola desde ${MY_HOSTNAME}</h1>
    <p>IP del servidor: ${MY_IP}</p>
    <p>Estado: Funcionando perfectamente</p>
  `);
});

server.listen(PORT, '0.0.0.0', () => {
  console.log(`Servidor corriendo en http://${MY_IP}:${PORT}`);
});

(20/33) Installing readline (8.2.13-r1)
(21/33) Installing sqlite-libs (3.49.2-r1)
(22/33) Installing python3 (3.12.13-r0)
(23/33) Installing python3-pycache-pyc0 (3.12.13-r0)
(24/33) Installing pyc (3.12.13-r0)
(25/33) Installing py3-setuptools-pyc (80.9.0-r0)
(26/33) Installing py3-pip-pyc (25.1.1-r0)
(27/33) Installing py3-parsing (3.2.5-r0)
(28/33) Installing py3-parsing-pyc (3.2.5-r0)
(29/33) Installing py3-packaging-pyc (25.0-r0)
(30/33) Installing python3-pyc (3.12.13-r0)
(31/33) Installing py3-packaging (25.0-r0)
(32/33) Installing py3-setuptools (80.9.0-r0)
(33/33) Installing py3-pip (25.1.1-r0)
Executing busybox-1.37.0-r20.trigger
OK: 222 MiB in 93 packages
webserver1:~# rc-update add nginx default
* service nginx added to runlevel default
webserver1:~# mkdir -p /var/www/python && cd /var/www/python
webserver1:~#
```

```
ssh janet@192.168.1.101
Warpify subshell Ctrl |

ssh janet@192.168.1.101
(14/17) Installing libxml2 (2.13.9-r0)
(15/17) Installing php83 (8.3.29-r0)
(16/17) Installing acl-libs (2.3.2-r1)
(17/17) Installing php83-fpm (8.3.29-r0)
Executing busybox-1.37.0-r20.trigger
OK: 181 MiB in 78 packages
webserver2:~# rc-update add php-fpm83 default
* service php-fpm83 added to runlevel default
webserver2:~# rc-update add nginx default
* service nginx added to runlevel default
webserver2:~# rc-service php-fpm83 start
* Caching service dependencies ...
* Starting nginx ...
* Checking /etc/php83/php-fpm.conf ...
* /run/php-fpm83: creating directory
* Starting PHP FastCGI Process Manager ...
webserver2:~# nano /etc/nginx/http.d/default.conf
webserver2:~# nano /var/www/localhost/htdocs/index.php
webserver2:~# rc-service php-fpm83 restart
* Stopping PHP FastCGI Process Manager ...
* Checking /etc/php83/php-fpm.conf ...
* Starting PHP FastCGI Process Manager ...
webserver2:~# rc-service nginx restart
* Stopping nginx ...
* Starting nginx ...
webserver2:~#
```

```
ssh janet@192.168.1.101
Warpify subshell Ctrl |

ssh janet@192.168.1.101
(11/17) Installing libcurl (8.14.1-r2)
(8/17) Installing curl (8.14.1-r2)
(9/17) Installing pcre2 (10.46-r0)
(10/17) Installing nginx (1.28.3-r0)
Executing nginx-1.28.3-r0.pre-install
Executing nginx-1.28.3-r0.post-install
(11/17) Installing nginx-openrc (1.28.3-r0)
(12/17) Installing php83-common (8.3.29-r0)
(13/17) Installing argon2-libs (20190702-r5)
(14/17) Installing libxml2 (2.13.9-r0)
(15/17) Installing php83 (8.3.29-r0)
(16/17) Installing acl-libs (2.3.2-r1)
(17/17) Installing php83-fpm (8.3.29-r0)
Executing busybox-1.37.0-r20.trigger
OK: 181 MiB in 78 packages
webserver2:~# rc-update add php-fpm83 default
* service php-fpm83 added to runlevel default
webserver2:~# rc-update add nginx default
* service nginx added to runlevel default
webserver2:~# rc-service php-fpm83 start
* Caching service dependencies ...
* Starting nginx ...
* Checking /etc/php83/php-fpm.conf ...
* /run/php-fpm83: creating directory
* Starting PHP FastCGI Process Manager ...
webserver2:~# nano /etc/nginx/http.d/default.conf
webserver2:~#
```

4. Pruebas De Conectividad (Ping Entre Máquinas)

Se verificó la conectividad entre todas las máquinas del entorno mediante el comando ping. Esto garantiza que la red está correctamente configurada antes de proceder con las pruebas de balanceo.

```
luisf@proxy:~$ ping -c 4 192.168.1.102

PING 192.168.1.102 (192.168.1.102) 56(84) bytes of data.
64 bytes from 192.168.1.102: icmp_seq=1 ttl=64 time=2.32 ms
64 bytes from 192.168.1.102: icmp_seq=2 ttl=64 time=2.62 ms
64 bytes from 192.168.1.102: icmp_seq=3 ttl=64 time=2.31 ms
^C
--- 192.168.1.102 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 2.314/2.417/2.615/0.140 ms

luisf@proxy:~$ ping -c 4 192.168.1.101

PING 192.168.1.101 (192.168.1.101) 56(84) bytes of data.
64 bytes from 192.168.1.101: icmp_seq=1 ttl=64 time=5.83 ms
64 bytes from 192.168.1.101: icmp_seq=2 ttl=64 time=2.89 ms
64 bytes from 192.168.1.101: icmp_seq=3 ttl=64 time=2.93 ms
64 bytes from 192.168.1.101: icmp_seq=4 ttl=64 time=2.66 ms

--- 192.168.1.101 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 2.664/3.579/5.833/1.304 ms
```

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Servidor wepserver1</title>
</head>
<body>
  <h1>👋 Python JHON CHRISTIAN | Servidor: wepserver1 | IP: 127.0.0.1</h1>
  <p>Tecnología: Python3 (http.server)</p>
</body>
</html> * status: started
```

```
GNU nano 8.4 app.py
from http.server import HTTPServer, BaseHTTPRequestHandler
import socket
import subprocess

hostname = socket.gethostname()
hostip = subprocess.check_output(['hostname', '-i']).decode().strip().split()[0]

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        html = f'''<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Servidor {hostname}</title>
</head>
<body>
  <h1>👋 Python JHON CHRISTIAN | Servidor: {hostname} | IP: {hostip}</h1>
  <p>Tecnología: Python3 (http.server)</p>
</body>
</html>'''
        self.send_response(200)
        self.send_header('Content-Type', 'text/html; charset=utf-8')
        self.end_headers()
        self.wfile.write(html.encode())

def log_message(self, format, *args):
    pass # Silencia logs innecesarios

if __name__ == '__main__':
    server = HTTPServer(('127.0.0.1', 5000), Handler)
    print(f'Servidor Python corriendo en http://127.0.0.1:5000/')

Help      Write Out  Where Is  Cut       Execute
Exit      Read File  Replace   Paste     Justify
ctrl-shift-w to use agent
```


5. Pruebas De Balanceo (Round Robin, Least Connection, IP Hash)

Round Robin es el algoritmo predeterminado de NGINX. Distribuye las peticiones de forma secuencial y equitativa entre todos los servidores del grupo upstream, siguiendo un orden circular: Backend1 → Backend2 → Backend3 → Backend1

```
luisf@proxy:~$ for i in {1..10}; do
  echo "Petición $i:"
  curl -s http://localhost | grep "<h1>"
done
Petición 1:
<h1> PHP | Servidor: webserver2 | IP: 192.168.1.101</h1>
Petición 2:
<h1> PHP | Servidor: webserver2 | IP: 192.168.1.101</h1>
Petición 3:
<h1> Python | Servidor: wepserver1 | IP: 127.0.0.1</h1>
Petición 4:
<h1> PHP | Servidor: RESPALDO webserver2 | IP: 192.168.1.102</h1>
Petición 5:
<h1> Python JHON CHRISTIAN | Servidor: wepserver1 | IP: 127.0.0.1</h1>
Petición 6:
<h1> PHP | Servidor: webserver2 | IP: 192.168.1.101</h1>
Petición 7:
<h1> Python | Servidor: wepserver1 | IP: 127.0.0.1</h1>
Petición 8:
<h1> PHP | Servidor: RESPALDO webserver2 | IP: 192.168.1.102</h1>
Petición 9:
<h1> Python JHON CHRISTIAN | Servidor: wepserver1 | IP: 127.0.0.1</h1>
Petición 10:
<h1> PHP | Servidor: webserver2 | IP: 192.168.1.101</h1>

luisf@proxy:~$ sudo nano /etc/nginx/sites-available/default

luisf@proxy:~$ Run commands

ctrl-shift-⌘ new /agent conversation
```

6. Simulación De Fallos

Para validar la tolerancia a fallos del sistema, se procedió a detener el servicio NGINX en uno de los servidores backend mientras el proxy continuaba recibiendo peticiones. NGINX detecta automáticamente el servidor caído y redirige el tráfico a los servidores disponibles.

Análisis De Resultados:

1. Comparación Entre Algoritmos De Equilibrio

Round Robin es el algoritmo más simple de los tres. Distribuye las peticiones de forma secuencial y equitativa entre todos los servidores disponibles, sin considerar el estado actual de cada uno. Como se observó en las pruebas, cada servidor recibe peticiones en rotación sin importar si alguno está más ocupado que otro. Esto lo hace ideal cuando los servidores tienen capacidades similares y las peticiones tienen tiempos de procesamiento parecidos, pero puede volverse ineficiente cuando las cargas son heterogéneas, ya que un servidor lento seguirá recibiendo peticiones aunque esté saturado.

Least Connection mejora este comportamiento al tomar decisiones dinámicas. En lugar de seguir un orden fijo, el proxy dirige cada nueva petición al servidor que en ese momento tenga el menor número de conexiones activas. Esto lo hace más inteligente ante cargas variables, porque evita acumular peticiones en servidores que ya están procesando tareas largas. La desventaja es que requiere que el proxy lleve un seguimiento constante del estado de las conexiones, lo que añade un pequeño

overhead de procesamiento, aunque en la práctica este costo es insignificante frente al beneficio que aporta en entornos con tráfico real.

IP Hash adopta un enfoque completamente distinto: en lugar de balancear la carga de forma dinámica, calcula un hash basado en la dirección IP del cliente y siempre dirige sus peticiones al mismo servidor. Esto garantiza persistencia de sesión sin necesidad de cookies ni bases de datos compartidas, lo cual es fundamental para aplicaciones que almacenan el estado del usuario en memoria del servidor. Sin embargo, su debilidad está en que la distribución de carga puede volverse desigual si muchos clientes comparten una misma IP pública (como ocurre detrás de un NAT corporativo), ya que todos irían al mismo backend.

2. Ventajas y Desventajas De Cada Algoritmo

Round Robin

Ventajas	Desventajas
Implementación simple y sin overhead	No considera la carga real de cada servidor
Distribución equitativa garantizada	Ineficiente si las peticiones tienen tiempos variables
Predecible y fácil de depurar	No hay persistencia de sesión
Adecuado para servidores homogéneos	Puede generar desbalance en escenarios reales

Least Connection

Ventajas	Desventajas
Adapta la distribución a la carga real	Mayor complejidad de implementación
Óptimo para peticiones de duración variable	Requiere seguimiento de conexiones activas
Evita sobrecargar servidores ocupados	Sin persistencia de sesión
Mejor aprovechamiento de recursos	Clientes con IP dinámica pierden afinidad

IP Hash

Ventajas	Desventajas
Garantiza persistencia de sesión	Distribución puede ser desigual
Ideal para aplicaciones con estado	Clientes con IP dinámica pierden afinidad
Consistente y predecible por cliente	Usuarios detrás de NAT van todos al mismo servidor
No requiere cookies de sesión	Dificulta escalar horizontalmente

3. Rendimiento Del Sistema En Cada Escenario

Durante las pruebas realizadas se observaron los siguientes comportamientos en términos de rendimiento y distribución de carga:

Escenario 1: Carga uniforme (Round Robin)

Cuando todas las peticiones tienen un tiempo de procesamiento similar, Round Robin demostró una distribución perfectamente equitativa. Los tres backends recibieron aproximadamente el mismo número de peticiones (33% cada uno). Este es el escenario ideal para este algoritmo

Escenario 2: Carga variable (Least Connection)

Al simular peticiones de duración heterogénea (algunas conexiones largas y otras

cortas), Least Connection demostró su superioridad al evitar dirigir nuevas peticiones a backends ya saturados. El balanceo dinámico resultó en tiempos de respuesta más consistentes y menor latencia promedio

Escenario 3: Persistencia de sesión (IP Hash)

Con IP Hash se verificó que todas las peticiones provenientes de un mismo cliente siempre fueron dirigidas al mismo backend. Esto es crítico para aplicaciones que almacenan información de sesión en memoria del servidor y que fallarían si el cliente fuera redirigido a otro servidor.

Conclusiones

Este laboratorio me permitió comprender en profundidad cómo funciona un proxy inverso en un entorno real. La configuración de NGINX como balanceador de carga demostró ser sencilla pero poderosa. La principal lección fue entender que no existe un único algoritmo 'mejor': la elección depende del tipo de aplicación y de los patrones de tráfico. En producción, optaría por Least Connection para APIs REST con tiempos variables, y consideraría IP Hash únicamente cuando la aplicación requiere estado de sesión sin una capa de caché compartida.

Anexos

